PART1: Building and Evaluating an MLP on MNIST Report:

1. Objective

The goal of this notebook was to implement, train, fine-tune and evaluate a simple multi-layer perceptron (MLP) classifier on the MNIST handwritten-digit dataset, and then explore how key hyperparameters affect its performance.

2. Dataset Preparation

- I downloaded the full MNIST training set (60 000 images) and split it into 42 000 training, 12 000 validation, and 6 000 test samples.
- For the training split we applied a small rotation (±10°) and a random horizontal flip, followed by conversion to a tensor and normalization (mean = 0.1307, std = 0.3081).
- Validation and test sets only used tensor conversion and normalization.
- DataLoaders were created with batch size = 32, shuffling on the training loader, and pinned memory to speed up transfers to GPU.

3. Model Definition

- The MLP has two hidden layers:
    - Input 784 → 256 neurons, followed by batch-norm, ReLU and dropout (p = 0.2).
    - Then 256 → 128 neurons, again with batch-norm, ReLU and dropout.
    - Finally a Linear layer producing 10 outputs (one per digit class).
- Cross-entropy loss and SGD with learning rate 0.01 and momentum 0.9 were chosen.

4. Device Configuration

- I ran everything on a T4 GPU in Colab.
- The model and data batches were explicitly moved to the GPU device.

5. Initial Training & Evaluation

5.1 Training Loop

- Trained for 5 epochs.
- After each epoch we recorded the average training loss and measured test accuracy.

5.2 **Learning Curves and Parameter Counts**

- The training-loss curve steadily decreased from around 0.49 down to 0.23.
- Test accuracy rose from ~92.6% in epoch 1 to ~95.1% by epoch 4–5.
- I counted 235 136 weight parameters and 778 bias parameters in total.

5.3 **Bias Parameters Before vs. After Training**

- **Initial biases** of the last Linear layer (fc3.bias):

[0.0172, 0.0158, -0.0608, -0.0645, -0.0478, -0.0545, -0.0488, 0.0354, 0.0829, -0.0618]

- **Final biases** after 5 epochs:

[-0.0394, -0.0422, -0.0340, -0.0864, -0.1561, -0.0266, -0.1354, -0.0095, 0.2429, 0.0997]

Observation: Every bias shifted during training—some by a few hundredths and others by over a tenth—showing that the model adjusted its offset terms significantly as it learned to better separate the digit classes.

6. Model Persistence and Final Test Metrics

- I saved the trained model's state and reloaded it successfully.
- On the 6 000-image test set, the final model achieved 95.30% accuracy with per-class precision, recall and F1-scores all above 88%, and most above 95%.

7. Hyperparameter Tuning (with Early Stopping)

I ran five separate sweeps—varying one parameter at a time, each with patience = 2 on validation loss:

- **Batch Size {4, 8, 16}:** Best at 16 (95.43% test accuracy).
- **Learning Rate {0.1, 0.01, 0.001}:** Best at 0.01 (95.80% test accuracy).
- **Number of Hidden Layers {2, 3, 4}:** Best at 3 (95.78% test accuracy).
- **Activation Function {ReLU, LeakyReLU, GELU}:** Best at GELU (95.87% test accuracy).
- **Optimizer {SGD, RMSprop, Adam}:** Best at SGD (95.60% test accuracy).

8. Final Model Training & Device Comparison
Using the best settings (batch = 16, lr = 0.01, three hidden layers of size 256 with GELU and dropout, and SGD), I:

- **On GPU (T4):**
    - 5 epochs yielded 95.82% test accuracy.
    - Total training time: **122.41 s**.
    - Final F1-weighted average: **95.76%**.
- **On CPU:**
    - Same model got 95.95% accuracy in 118.22 s.

Surprisingly, the CPU run was slightly faster here—likely because the model is small, and data-transfer overhead to GPU outweighs its compute advantage. In larger networks, the GPU payoff would be more dramatic.

Conclusion
Notebook 1 walks through a full deep-learning workflow: data loading and augmentation, model design with regularization, training loops with timing, metrics reporting, model saving/loading, and systematic hyperparameter tuning with early stopping. The bias comparison confirms that the network learned nontrivial offsets in its final layer, and our final model achieves nearly 96% accuracy on MNIST, demonstrating the effectiveness of this simple MLP when carefully tuned.

PART2: Backpropagation Notebook Report:

Comparing SGD vs. Adam

Part (i): Repeat e–g with Adam. Which optimizer does a better job, and why?

- **Adam Setup**: same network, optimizer = Adam(lr=0.001), 10 epochs.
- **Training with Adam**:

Epoch 1 → Loss: 0.2505, Acc: 92.76%

…

Epoch 10 → Loss: 0.0186, Acc: 99.38%

- **Test Accuracy (Adam)**: **97.62%** (same as SGD)

Comparison & Discussion

- Both optimizers reach **the same final test accuracy** on MNIST (97.62%).
- **Convergence Speed**
    - **SGD+momentum** reached 99.61% train accuracy by epoch 10, slightly ahead of Adam's 99.38%.
    - **Adam** started off a bit slower but quickly caught up.
- **Why?**
    - On a small, well-behaved dataset like MNIST and a simple MLP, both methods can find a good minimum.
    - **SGD+momentum** leverages consistent gradient directions for stable, fast bulk progress.
    - **Adam** adapts learning rates per-parameter, helping in settings with sparse or noisy gradients—but here the task is so smooth that adaptive steps aren't a big advantage.
- **Takeaway**
    - For larger or more complex models (e.g., deep CNNs, NLP transformers), Adam often yields faster or more robust convergence "out of the box."
    - For simpler networks and when you can tune the learning rate, SGD+momentum remains a strong, reliable choice.
    -

Conclusion

- I successfully implemented and tested a custom NumPy backprop system.
- I built a small PyTorch MLP that hits ~97.6% test accuracy on MNIST.
- Both SGD and Adam perform equally well on this task, with SGD showing a slight edge in raw training speed.
- This demonstrates both the mechanics of backprop at the math level and the practical interplay of different optimizers in PyTorch.